Light

Computational Thinking

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Abstraction
2. Decomposition
3. Algorithmic thinking
Worked examples
Quick reference
Typical pitfalls

OCR GCSE Computer Science — **Computational Thinking (J277/02 Section 2.1)**. This topic covers the fundamental problem-solving approaches used in computer science: abstraction (removing unnecessary detail), decomposition (breaking problems into smaller parts), and algorithmic thinking (creating step-by-step solutions). These skills are essential not just for programming, but for solving problems in any field.

Exam questions typically ask you to **explain** each computational thinking technique, **apply** them to given scenarios, **identify** which technique is being used, and **demonstrate** how you would use these approaches to solve problems. The 2024 exam included questions requiring candidates to explain benefits of decomposition and identify abstraction in real-world contexts. Expect questions worth 2-6 marks that test your understanding of all three techniques.

Why computational thinking matters

Computational thinking is not just for computer scientists — it's a universal problem-solving approach that can be applied to any complex challenge. Whether you're planning an event, writing an essay, or building a website, these techniques help you:

- **Tackle complexity:** Large problems become manageable when broken into smaller parts
- **Focus on what matters:** Filtering out irrelevant detail helps you see the core problem clearly
- **Create reliable solutions:** Step-by-step algorithms can be tested, refined, and reused
- **Communicate clearly:** Structured approaches make it easier to explain your thinking to others
- **Work efficiently:** Systematic approaches prevent wasted effort on the wrong things

In the exam, you will be expected to apply these techniques to unfamiliar scenarios. The key is understanding the principles well enough to adapt them to any situation.

What you must know

This checklist covers the key exam-visible concepts. You should be able to explain each point and apply it to real-world scenarios.

- **Define** computational thinking and explain why it is important for problem solving.
- **Explain** abstraction and give examples of removing unnecessary detail.
- **Identify** what details to include and exclude when abstracting a problem.
- **Explain** decomposition and demonstrate breaking a problem into sub-problems.
- **Apply** decomposition to given scenarios (e.g., planning a school event, creating a game).
- **Explain** algorithmic thinking and the process of creating step-by-step solutions.
- **Create** simple algorithms using pseudocode or flowcharts.
- **Identify** which computational thinking technique is being applied in a given scenario.
- **Explain** the benefits of using each technique.

> **Specification note.** OCR J277 Section 2.1 covers computational thinking principles. Questions often present a scenario and ask you to explain how abstraction, decomposition, or algorithmic thinking would be applied. Being able to give specific, contextual examples is essential for full marks.

How the three techniques work together

In real problem-solving, all three techniques are used together:

1. **First, use decomposition** to break the large problem into manageable sub-problems
2. **Then, use abstraction** to identify the essential information needed for each sub-problem
3. **Finally, use algorithmic thinking** to create step-by-step solutions for each sub-problem

Example — Building a traffic light simulation:

- **Decomposition:** Break into: light display, timing system, pedestrian button, emergency override
- **Abstraction:** Focus on light colours and timing; ignore the physical construction of the lights or electrical wiring
- **Algorithmic thinking:** Create sequences like: red (30 secs) → red+amber (5 secs) → green (30 secs) → amber (5 secs) → repeat

Toolkit

The three pillars of computational thinking

Technique	What it means	Key question to ask
Abstraction	Removing unnecessary detail to focus on what's important	"What can I ignore?"
Decomposition	Breaking a complex problem into smaller, manageable parts	"How can I split this up?"
Algorithmic thinking	Creating a step-by-step solution that can be followed	"What are the exact steps?"

Real-world examples at a glance

Technique	Everyday example	Computing example
Abstraction	A tube map shows stations and lines but ignores street layouts, buildings, and exact distances	A variable name like "playerScore" hides how the number is stored in binary
Decomposition	Planning a party: venue, invitations, food, music, decorations as separate tasks	Creating a game: input handling, game logic, graphics, sound as separate modules
Algorithmic thinking	A recipe with step-by-step instructions for baking a cake	A sorting algorithm that puts a list of numbers in order

Benefits of computational thinking

Abstraction benefits:

- Focuses attention on important details
- Makes problems easier to understand
- Reduces complexity
- Creates reusable solutions

Decomposition benefits:

- Makes large problems manageable
- Allows team collaboration
- Easier to test individual parts
- Identifies all components needed

Algorithmic thinking benefits:

- Creates clear, repeatable solutions
- Solutions can be automated
- Easier to identify errors
- Solutions can be shared and reused

> **Exam tip:** When asked to explain a benefit, always relate it to the CONTEXT of the question. "It makes the problem easier" is too vague. "It allows the team to work on different parts simultaneously, reducing development time" is specific and earns marks.

1. Abstraction

Abstraction is the process of removing unnecessary detail from a problem to focus on the essential information needed to solve it. It involves identifying what is relevant and filtering out what is not.

Abstraction does not mean ignoring details permanently — it means temporarily setting aside details that are not relevant to the current problem. The removed details might be important for other purposes, but they would distract from solving the immediate problem.

1.1 What abstraction involves

When applying abstraction, you need to:

- **Identify essential information:** What details are absolutely necessary to solve the problem?
- **Remove unnecessary detail:** What information can be ignored without affecting the solution?
- **Focus on the problem:** Keep only what is relevant to the task at hand.
- **Create a simplified model:** Represent the problem in a way that highlights what matters.

1.2 Everyday examples of abstraction

Abstraction is everywhere in daily life. Whenever we use a simplified representation of something complex, we are using abstraction.

London Underground map:

- **Includes:** Station names, lines, connections between lines, zones
- **Excludes:** Actual distances, street layouts, buildings, geographical accuracy, tunnel depths
- **Why:** Passengers need to know how to get from A to B, not the exact geography. The simplified map makes journey planning quick and easy.

Weather forecast:

- **Includes:** Temperature, rain chance, wind speed, general conditions (sunny, cloudy, rainy)

- **Excludes:** Detailed atmospheric pressure data, satellite imagery, complex weather models, humidity at different altitudes
- **Why:** People need to know if they should bring an umbrella or wear a coat, not the science behind meteorological predictions.

Road signs:

- **Includes:** Simple symbols representing hazards, speed limits, or directions
- **Excludes:** Detailed explanations of why the hazard exists, historical accident data, engineering specifications
- **Why:** Drivers need quick, clear information they can process instantly while driving safely.

Contact list on a phone:

- **Includes:** Name, phone number, email address
- **Excludes:** Person's full biography, their daily schedule, their relationships with other contacts
- **Why:** The purpose is to contact people, so only contact information is needed.

School timetable:

- **Includes:** Subject, time, room number, teacher name
- **Excludes:** Detailed lesson plans, learning objectives for each lesson, seating arrangements
- **Why:** Students need to know where to be and when, not what will be taught in each lesson.

1.3 Abstraction in computing

Variables:

- A variable like `score = 100` abstracts away how the number 100 is stored in binary (01100100)
- The programmer works with meaningful names, not memory addresses

Functions/procedures:

- Using `print("Hello")` abstracts away the complex process of displaying characters on screen
- The programmer doesn't need to know how pixels are controlled

High-level programming languages:

- Python abstracts away machine code and memory management
- Programmers write readable code instead of binary instructions

User interfaces:

- A "Save" button abstracts away file system operations, data encoding, and disk writes
- Users click one button instead of understanding storage mechanisms

1.4 Applying abstraction to problems

When solving a problem, you must decide what information is essential and what can be ignored. This decision depends on the specific purpose of the solution.

Example: Creating a school registration system

What to **include**:

- Student name, year group, form class
- Attendance status (present/absent/late)
- Date and time of registration
- Unique student ID number

What to **exclude**:

- Student's favourite colour or hobbies
- What they had for breakfast
- Their journey to school or mode of transport
- Their height and weight (unless specifically required for PE)
- Their friendship groups or social relationships

Example: Creating an online shopping cart

What to **include**:

- Product name, price, quantity selected
- Product ID for inventory tracking
- Customer delivery address
- Payment method

What to **exclude**:

- Manufacturing process of each product
- Warehouse shelf location
- Delivery driver's name and route
- Customer's browsing history (unless used for recommendations)

> **Exam tip:** When asked what to include/exclude, think: "Does this help solve the specific problem?" If no, it should be excluded. Always justify your choices by explaining why each piece of information is or isn't relevant to the task.

Worked Example 1 – Explaining abstraction

Question: Explain what is meant by abstraction in computational thinking. [2 marks]

Model answer:
Abstraction is the process of removing/filtering out unnecessary detail from a problem **(1)**.
This allows you to focus on the essential information needed to solve the problem **(1)**.

> **Exam tip:** Two key parts: (1) removing detail, (2) focusing on what's important. Both parts needed for full marks.

Worked Example 2 – Identifying abstraction in a real-world example

Question: A London Underground map is an example of abstraction. Explain how abstraction has been applied in the design of this map. [4 marks]

Model answer:
The map includes only essential information such as station names and the lines connecting them **(1)**.
Unnecessary details have been removed, such as the actual geographical distances between stations **(1)**.

The map does not show street layouts, buildings, or the exact routes the tunnels take **(1)**.
This makes it easier for passengers to plan their journey without being distracted by irrelevant information **(1)**.

Exam tip: Give SPECIFIC examples of what is included AND excluded. Then explain WHY this helps users.

Worked Example 3 – Applying abstraction to a scenario

Question: A programmer is creating a library book lending system. Identify two pieces of information that should be included and two that should be excluded. Justify your choices. [4 marks]

Model answer:
Include — Book title: This is essential to identify which book is being borrowed **(1)**.
Include — Borrower's library card number: This is needed to track who has borrowed the book and when it should be returned **(1)**.
Exclude — Author's date of birth: This is not relevant to tracking whether a book is on loan or available **(1)**.
Exclude — Borrower's favourite genre: While interesting, this does not help manage the lending process **(1)**.

Exam tip: Always JUSTIFY each choice. Don't just list items — explain WHY they should be included or excluded.

2. Decomposition

Decomposition is the process of breaking down a complex problem or system into smaller, more manageable parts (sub-problems). Each sub-problem can then be solved individually, making the overall problem easier to tackle.
Decomposition is like breaking a large task into a to-do list. Instead of facing one overwhelming problem, you have several smaller tasks that can be completed one at a time or distributed among team members.

2.1 What decomposition involves

When applying decomposition, you:

- **Identify the main problem:** What is the overall goal or task?
- **Break it into parts:** What are the major components or stages?
- **Break parts further:** Can each part be broken down into even smaller tasks?
- **Identify dependencies:** Which parts depend on others being completed first?
- **Solve each part:** Address each sub-problem individually.

2.2 Everyday examples of decomposition

Planning a birthday party:

```
Birthday Party
├── Venue (choose location, book venue, arrange transport)
├── Guests (create guest list, send invitations, track RSVPs)
├── Food (plan menu, buy ingredients, prepare food)
├── Entertainment (choose activities, hire DJ, plan games)
└── Decorations (choose theme, buy supplies, set up)
```

Writing an essay:

```
Essay
├── Research (find sources, take notes, organise findings)
├── Planning (create outline, structure arguments)
├── Writing (introduction, main body, conclusion)
├── Editing (check spelling, improve clarity, verify references)
└── Submission (format correctly, save file, submit)
```

2.3 Decomposition in computing

Creating a computer game:

```
Computer Game
├── User interface (main menu, settings, pause menu)
├── Input handling (keyboard, mouse, controller)
├── Game logic (rules, scoring, win/lose conditions)
├── Graphics (sprites, backgrounds, animations)
├── Sound (music, sound effects, volume control)
└── Data storage (save games, high scores, settings)
```

Building a website:

```
Website
├── Design (layout, colours, fonts, images)
├── Content (text, images, videos)
├── Navigation (menu, links, buttons)
├── Functionality (forms, search, login)
└── Testing (browser compatibility, mobile responsiveness)
```

2.4 Benefits of decomposition

- **Manageable tasks:** Large, overwhelming problems become a series of smaller, achievable tasks that are easier to understand and complete
- **Team collaboration:** Different people can work on different parts simultaneously, speeding up development
- **Easier testing:** Each component can be tested independently before combining, making it easier to find and fix errors

- **Reusability:** Sub-solutions can often be reused in other projects, saving time in the future
- **Clearer understanding:** Breaking down reveals all the components needed, ensuring nothing is forgotten
- **Progress tracking:** Easier to measure progress when tasks are clearly defined and can be marked as complete
- **Maintenance:** If one part needs updating, it can be changed without affecting other parts
- **Documentation:** Decomposed projects are easier to document and explain to others

2.5 How to decompose a problem

Follow these steps when decomposing a problem:

1. **Understand the problem:** What is the overall goal? What inputs are needed? What outputs are expected?
2. **Identify major components:** What are the main parts or stages of the solution?
3. **Break down further:** Can each major component be split into smaller tasks?
4. **Consider dependencies:** Which tasks must be completed before others can start?
5. **Check completeness:** Have you identified everything needed to solve the problem?

Example — Decomposing a quiz program:

```
Quiz Program
├── Setup (load questions, initialise score, welcome message)
├── Question loop (display question, get answer, check answer, update score)
├── Scoring (calculate percentage, determine grade)
├── Results (display final score, show correct answers, offer retry)
└── Data storage (save high scores, load previous results)
```

Exam tip: When decomposing a problem in an exam, use a clear structure (like the tree diagrams above). Show at least TWO levels of breakdown to demonstrate thorough decomposition. Each sub-problem should be specific to the scenario given.

Worked Example 4 — Explaining decomposition

Question: Explain what is meant by decomposition in computational thinking. [2 marks]

Model answer:
Decomposition is the process of breaking down a complex problem into smaller, more manageable parts/sub-problems **(1)**.
Each smaller part can then be solved individually, making the overall problem easier to tackle **(1)**.

Exam tip: Key words: "breaking down", "smaller parts", "manageable", "sub-problems".

Worked Example 5 — Applying decomposition to a scenario

Question: A school wants to create a program to manage student attendance. Use decomposition to break this problem into smaller parts. [4 marks]

Model answer:
Student management: Add new students, edit student details, remove students who leave **(1)**.
Recording attendance: Mark students present or absent, record time of arrival, handle late arrivals **(1)**.
Reports: Generate attendance reports, calculate attendance percentages, identify patterns **(1)**.
User interface: Login system, navigation menu, input forms for data entry **(1)**.

Exam tip: Each sub-problem should be SPECIFIC to the scenario. Show that you've thought about what the system actually needs to do.

Worked Example 6 — Explaining benefits of decomposition

Question: A team of programmers is creating a new mobile app. Explain two benefits of using decomposition for this project. [4 marks]

Model answer:
Benefit 1: Different team members can work on different parts of the app simultaneously **(1)**. For example, one programmer can work on the user interface while another works on the database, reducing overall development time **(1)**.
Benefit 2: Each component can be tested independently before being combined **(1)**. This makes it easier to identify and fix bugs in specific sections rather than searching through the entire codebase **(1)**.

Exam tip: For benefit questions, state the benefit AND explain HOW/WHY it helps. Link your answer to the specific scenario (here: team of programmers, mobile app).

3. Algorithmic thinking

Algorithmic thinking is the process of developing a step-by-step solution to a problem. An algorithm is a precise sequence of instructions that, when followed, will solve a problem or complete a task.

Algorithmic thinking is about being specific and precise. Each step must be clear enough that anyone (or any computer) could follow it without needing to make decisions or interpretations.

3.1 What algorithmic thinking involves

When applying algorithmic thinking, you:

- **Define the problem:** What exactly needs to be achieved?
- **Identify inputs:** What information or data is needed?
- **Identify outputs:** What result should be produced?
- **Determine the steps:** What sequence of actions will transform inputs into outputs?
- **Be precise:** Each step must be clear and unambiguous.
- **Consider all cases:** What happens in different situations (e.g., errors, edge cases)?

3.2 Characteristics of a good algorithm

- **Precise:** Each instruction is clear and unambiguous
- **Finite:** The algorithm will eventually stop (not run forever)
- **Effective:** Each step can actually be carried out
- **Produces output:** The algorithm generates a result
- **General:** Works for all valid inputs, not just specific cases

3.3 Everyday algorithm examples

Making a cup of tea:

1. Fill kettle with water
2. Turn on kettle
3. Wait for kettle to boil
4. Place tea bag in cup
5. Pour boiling water into cup
6. Wait 3-5 minutes for tea to brew
7. Remove tea bag
8. Add milk and sugar if desired
9. Stir and serve

Crossing the road safely:

1. Stop at the kerb
2. Look right
3. Look left
4. Look right again
5. If road is clear, walk across
6. Otherwise, wait and repeat from step 2

Logging into a computer:

1. Turn on the computer
2. Wait for login screen to appear
3. Enter username
4. Enter password
5. Click login button
6. If credentials correct, desktop loads
7. If credentials incorrect, display error and return to step 3

> **Why order matters:** If you tried to pour water before boiling it, or remove the tea bag before adding it, the algorithm would fail. The sequence of steps is crucial — algorithms must be followed in the correct order to achieve the desired result.

3.4 Inputs, processes, and outputs

Every algorithm involves three key components:

- **Inputs:** The data or information that goes INTO the algorithm (e.g., two numbers to add)
- **Processes:** The operations performed on the inputs (e.g., adding the numbers together)
- **Outputs:** The result produced by the algorithm (e.g., the sum of the two numbers)

Example — Calculating a restaurant bill:

- **Inputs:** Cost of each item ordered, number of people sharing
- **Processes:** Add up all item costs, divide by number of people
- **Outputs:** Amount each person should pay

3.5 Representing algorithms

Algorithms can be represented in several ways. The most common methods at GCSE are pseudocode and flowcharts.

Pseudocode: Structured English that resembles programming code but is designed to be human-readable. It uses keywords like INPUT, OUTPUT, IF, WHILE, FOR to describe the logic of an algorithm without worrying about the exact syntax of a specific programming language.

```
INPUT number
IF number >= 0 THEN
    OUTPUT "Positive or zero"
ELSE
    OUTPUT "Negative"
ENDIF
```

Advantages of pseudocode:

- Language-independent — can be converted to any programming language
- Easier to write than actual code
- Focuses on logic rather than syntax
- Can include more detail than flowcharts for complex algorithms

Flowcharts: Visual diagrams using standard shapes to represent different types of operations. Arrows show the flow of control from one operation to the next.

Shape	Meaning	Example
Oval/Rounded rectangle	Start or End	Start, Stop
Parallelogram	Input or Output	Enter name, Display result
Rectangle	Process	Calculate total, Set counter to 0
Diamond	Decision	Is age >= 18? Yes/No
Arrows	Flow direction	Connect shapes to show sequence

Advantages of flowcharts:

- Visual representation makes logic easy to follow
- Good for showing the structure of decisions and loops
- Easier for non-programmers to understand
- Useful for planning before writing code

3.6 Types of constructs in algorithms

All algorithms are built from three fundamental building blocks called **constructs**:

Sequence: Steps performed one after another in a set order. This is the simplest construct — instructions are executed in the order they are written.

```
INPUT name
INPUT age
OUTPUT "Hello " + name
OUTPUT "You are " + age + " years old"
```

Selection: Making decisions based on conditions. The program chooses which path to follow depending on whether a condition is true or false. Uses IF...THEN...ELSE structures.

```
INPUT age
IF age >= 18 THEN
    OUTPUT "You can vote"
ELSE
    OUTPUT "You cannot vote yet"
ENDIF
```

Iteration (loops): Repeating a set of steps multiple times. There are different types of loops:

- **FOR loop:** Repeats a set number of times (count-controlled)
- **WHILE loop:** Repeats while a condition is true (condition-controlled)
- **REPEAT...UNTIL:** Repeats until a condition becomes true

```
// FOR loop - repeats exactly 5 times
FOR i = 1 TO 5
    OUTPUT i
NEXT i

// WHILE loop - repeats while condition is true
count = 1
WHILE count <= 5
    OUTPUT count
    count = count + 1
ENDWHILE
```

> **Exam tip:** When writing algorithms, be SPECIFIC. "Calculate the answer" is too vague. "Multiply the price by the quantity" is precise and clear. Examiners want to see that you understand exactly what operations are needed.

> Worked Example 7 — Explaining algorithmic thinking

Question: Explain what is meant by algorithmic thinking in computational thinking. [2 marks]

Model answer:
Algorithmic thinking is the process of developing a step-by-step solution to a problem **(1)**.
Each step must be precise and unambiguous so that it can be followed by a person or computer to solve the problem **(1)**.

Exam tip: Key words: "step-by-step", "sequence", "precise", "unambiguous".

Worked Example 8 — Writing an algorithm in pseudocode

Question: Write an algorithm in pseudocode that asks the user for two numbers and outputs the larger of the two. [4 marks]

Model answer:

```
INPUT num1
INPUT num2
IF num1 > num2 THEN
    OUTPUT num1
ELSE
    OUTPUT num2
ENDIF
```

Marking: Input both numbers **(1)**, correct comparison **(1)**, output larger number **(1)**, correct structure/syntax **(1)**.

Exam tip: Use clear variable names. Use proper pseudocode keywords (INPUT, OUTPUT, IF, THEN, ELSE, ENDIF). Indent code inside IF statements.

Worked Example 9 — Describing flowchart symbols

Question: State what each of the following flowchart shapes represents: (a) diamond, (b) parallelogram, (c) rectangle. [3 marks]

Model answer:
(a) Diamond represents a **decision** — a point where the algorithm chooses between two or more paths based on a condition **(1)**.
(b) Parallelogram represents **input or output** — where data is entered or results are displayed **(1)**.

(c) Rectangle represents a **process** — an action or calculation being performed **(1)**.

Exam tip: Learn all five flowchart shapes. The diamond (decision) and parallelogram (input/output) are most commonly tested.

Worked Example 10 – Writing an algorithm with iteration

Question: Write an algorithm in pseudocode that outputs the numbers 1 to 10. [3 marks]

Model answer:

```
FOR counter = 1 TO 10
    OUTPUT counter
NEXT counter
```

Alternative using WHILE:

```
counter = 1
WHILE counter <= 10
    OUTPUT counter
    counter = counter + 1
ENDWHILE
```

Marking: Correct loop structure **(1)**, correct range (1 to 10) **(1)**, correct output **(1)**.

Exam tip: Either FOR or WHILE is acceptable. Make sure you include the loop termination condition and increment the counter if using WHILE.

Worked Example 11 – Identifying computational thinking techniques

Question: For each scenario below, identify which computational thinking technique is being used. [3 marks]
(a) A programmer divides a game into modules: graphics, sound, and user input.
(b) A tube map shows only stations and lines, not actual distances.
(c) A recipe provides numbered steps to bake a cake.

Model answer:
(a) **Decomposition** — the problem (creating a game) has been broken into smaller parts (modules) **(1)**.
(b) **Abstraction** — unnecessary detail (distances, streets) has been removed to focus on essential information **(1)**.

(c) **Algorithmic thinking** — a step-by-step sequence of instructions has been created **(1)**.

Exam tip: Look for clues: "break into parts" = decomposition, "remove detail" = abstraction, "steps/sequence" = algorithmic thinking.

Worked Example 12 – Combining all three techniques

Question: A school wants to create a system to calculate and display student report grades. Explain how abstraction, decomposition, and algorithmic thinking could be used to develop this system. [6 marks]

Model answer:
Abstraction: Focus only on essential information such as student name, subject, and marks **(1)**. Exclude irrelevant details like the student's hobbies or favourite colour **(1)**.
Decomposition: Break the system into smaller parts such as: input module (entering marks), calculation module (working out grades), and output module (displaying the report) **(1)**. Each module can be developed and tested separately **(1)**.
Algorithmic thinking: Create step-by-step instructions for calculating grades, such as: input marks, calculate average, determine grade based on average, display result **(1)**. This could be written as pseudocode or a flowchart that can be converted to program code **(1)**.

Exam tip: For extended questions, address ALL THREE techniques. Give SPECIFIC examples from the scenario for each technique.

Worked examples summary

The 12 worked examples above cover the main question types you will encounter on computational thinking. Here is a summary of the key question patterns:

Example	Question type	Key skill
1	Explain abstraction (2 marks)	Definition: removing detail + focusing on important
2	Identify abstraction in example (4 marks)	What's included, excluded, and why
3	Apply abstraction to scenario (4 marks)	Include/exclude with justification

Example	Question type	Key skill
4	Explain decomposition (2 marks)	Definition: breaking into smaller parts
5	Apply decomposition to scenario (4 marks)	Identify specific sub-problems
6	Explain benefits of decomposition (4 marks)	Benefit + explanation in context
7	Explain algorithmic thinking (2 marks)	Definition: step-by-step, precise
8	Write algorithm — selection (4 marks)	IF...THEN...ELSE structure
9	Flowchart symbols (3 marks)	Diamond, parallelogram, rectangle meanings
10	Write algorithm — iteration (3 marks)	FOR or WHILE loop structure
11	Identify technique from scenario (3 marks)	Match clues to correct technique
12	Apply all three techniques (6 marks)	Abstraction + decomposition + algorithmic

Quick reference

Key terms – exam-ready definitions

Term	Definition (mark scheme style)
Computational thinking	A set of problem-solving techniques used to analyse and solve problems in a way that can be implemented by a computer
Abstraction	Removing unnecessary detail from a problem to focus on essential information
Decomposition	Breaking down a complex problem into smaller, more manageable parts/sub-problems
Algorithmic thinking	Developing a step-by-step solution to a problem; creating a precise sequence of instructions
Algorithm	A sequence of steps or instructions that, when followed, will solve a problem or complete a task
Pseudocode	A way of writing algorithms using structured English that resembles programming code

Term	Definition (mark scheme style)
Flowchart	A visual diagram that represents an algorithm using standard shapes and arrows
Sequence	Steps performed one after another in a set order
Selection	A decision point where different paths are taken based on a condition (IF...THEN...ELSE)
Iteration	Repeating a set of steps multiple times (loops: FOR, WHILE, REPEAT)

Flowchart symbols

Shape	Name	Purpose
Oval / Rounded rectangle	Terminator	Start or End of the algorithm
Parallelogram	Input/Output	Data input or output
Rectangle	Process	Action or calculation
Diamond	Decision	Yes/No question; branches the flow
Arrow	Flow line	Direction of flow between shapes

Pseudocode keywords

Keyword	Purpose	Example
INPUT	Get data from user	INPUT name
OUTPUT / PRINT	Display result	OUTPUT "Hello"
IF...THEN...ELSE...ENDIF	Selection/decision	IF age >= 18 THEN OUTPUT "Adult" ENDIF
FOR...TO...NEXT	Count-controlled loop	FOR i = 1 TO 10 ... NEXT i
WHILE...ENDWHILE	Condition-controlled loop	WHILE x < 100 ... ENDWHILE
REPEAT...UNTIL	Loop until condition true	REPEAT ... UNTIL answer = "yes"

Identification clues

If you see...	The technique is...
Removing detail, simplifying, ignoring, focusing on important parts	**Abstraction**
Breaking down, splitting up, dividing into parts, modules, sub-problems	**Decomposition**
Step-by-step, sequence, instructions, recipe, algorithm, flowchart	**Algorithmic thinking**

Typical pitfalls

Pitfall 1 — Confusing abstraction and decomposition.
Wrong: "Abstraction is breaking a problem into parts."
Right: Abstraction = removing unnecessary detail. **Decomposition** = breaking into smaller parts. They're different techniques!

Pitfall 2 — Vague abstraction answers.
Wrong: "Remove stuff you don't need."
Right: "Remove unnecessary detail such as [specific example] to focus on essential information like [specific example]."

Pitfall 3 — Not justifying include/exclude choices.
Wrong: "Include: name, age. Exclude: height."
Right: "Include name because it identifies the student. Exclude height because it is not relevant to tracking attendance."

Pitfall 4 — Shallow decomposition.
Wrong: Breaking problem into only 2 very general parts.
Right: Show multiple levels of breakdown with specific, meaningful sub-problems.

Pitfall 5 — Generic benefit explanations.
Wrong: "It makes it easier."
Right: "It allows team members to work on different modules simultaneously, reducing development time."

Pitfall 6 — Vague algorithm steps.
Wrong: "Calculate the thing."
Right: "Multiply price by quantity to calculate total cost."

Pitfall 7 — Missing pseudocode structure.

Wrong: Forgetting ENDIF, ENDWHILE, or NEXT.
Right: Always close your structures: IF needs ENDIF, WHILE needs ENDWHILE, FOR needs NEXT.

Pitfall 8 — Wrong flowchart shapes.
Wrong: Using a rectangle for input.
Right: Parallelogram = input/output, Rectangle = process, Diamond = decision.

Pitfall 9 — Not relating to context.
Wrong: Generic explanation without mentioning the scenario.
Right: Use specific details from the question in your answer.

Pitfall 10 — Forgetting to address all parts.
Wrong: Only explaining one technique when asked about all three.
Right: Check the question — if it asks for abstraction, decomposition, AND algorithmic thinking, cover all three.

Assessment Hub | Long-Answer | Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.